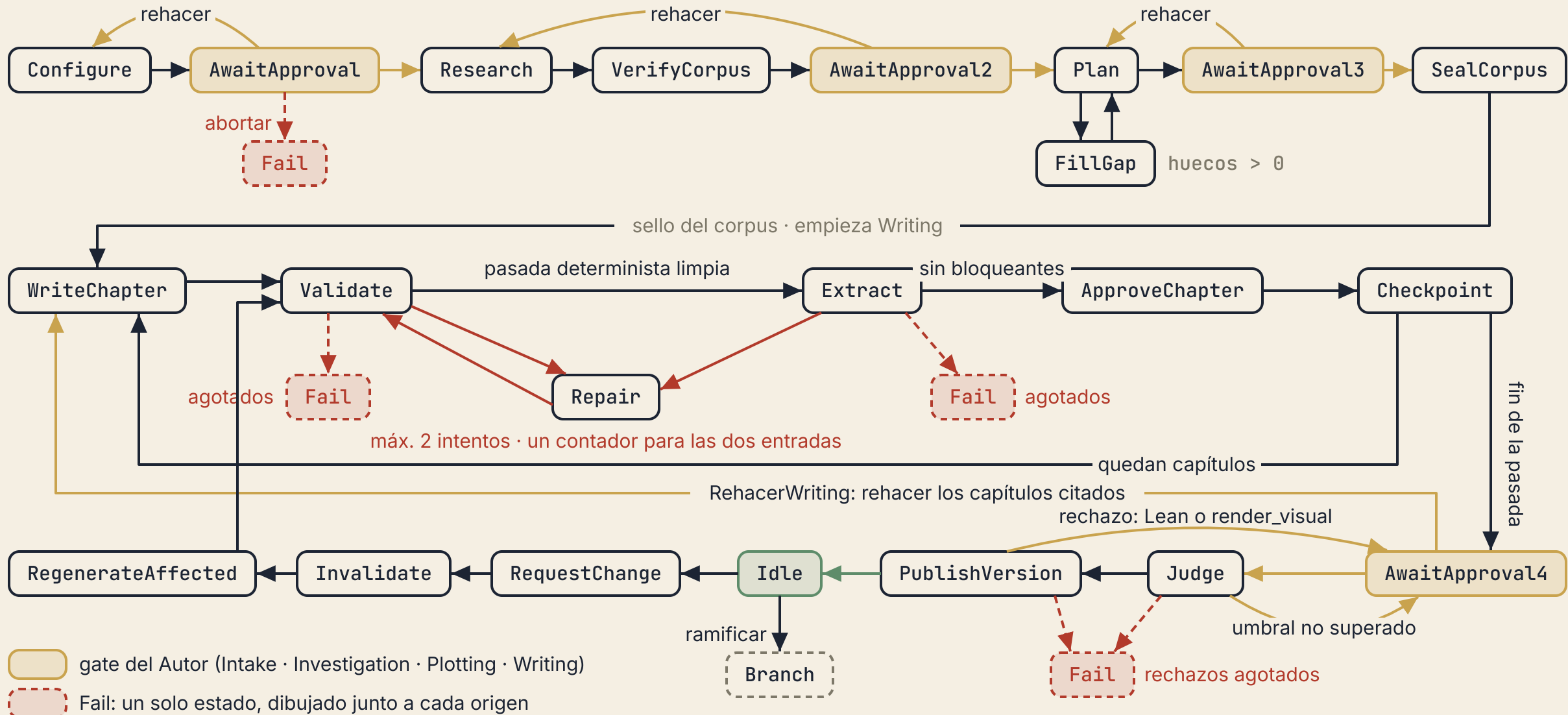


El arnés tiene 24 estados y 38 aristas, y ninguna acción puede mover el pc fuera de ellas



Las transiciones se declaran una sola vez, y de ahí sale el Next que TLC explora

```
\* Mover el pc es siempre lo mismo:
\* comprobar que la arista existe.
```

```
Mueve(de, a) ==
  /\ vivo
  /\ pc = de
  /\ <<de, a>> \in Aristas
  /\ pc' = a
```

Cuatro condiciones y nada más. El proceso está vivo, el pc está en el origen, el par <<de, a>> pertenece a Aristas y el pc pasa al destino. Cada acción añade sus guardas y su UNCHANGED.

Aristas gobierna, no acompaña. Como toda acción pasa por Mueve, TLC no puede explorar una transición que no esté en el conjunto.

La prueba lee la misma relación.

identidad_nodo_accion compara los 24 nodos y la relación ARISTAS de commons/graph/aristas.py con esta definición. Bloquea en G1.

Reintentar no es una arista: escribe el checkpoint «como salida de SealCorpus» y exige SealCorpus → WriteChapter.

```
Aristas ==
{ \* Fase 1 · Intake
  <<"Configure", "AwaitApproval">>,
  <<"AwaitApproval", "Research">>,
  <<"AwaitApproval", "Configure">>,
  <<"AwaitApproval", "Fail">>,
  \* Fase 2 · Investigation
  <<"Research", "VerifyCorpus">>,
  <<"VerifyCorpus", "AwaitApproval2">>,
  <<"AwaitApproval2", "Plan">>,
  <<"AwaitApproval2", "Research">>,
  \* Fase 3 · Plotting
  <<"Plan", "FillGap">>,
  <<"FillGap", "Plan">>,
  <<"Plan", "AwaitApproval3">>,
  <<"AwaitApproval3", "SealCorpus">>,
  <<"AwaitApproval3", "Plan">>,
  \* Fase 4 · Writing
  <<"SealCorpus", "WriteChapter">>,
  <<"WriteChapter", "Validate">>,
  <<"Validate", "Repair">>,
  <<"Validate", "Extract">>,
  <<"Extract", "Repair">>,
  <<"Repair", "Validate">>,
  <<"Validate", "Fail">>
}
```

```
<<"Extract", "Fail">>,
<<"Extract", "ApproveChapter">>,
<<"ApproveChapter", "Checkpoint">>,
<<"Checkpoint", "WriteChapter">>,
<<"Checkpoint", "AwaitApproval4">>,
\* Fase 5 · Publication
<<"AwaitApproval4", "Judge">>,
<<"AwaitApproval4", "WriteChapter">>,
<<"Judge", "PublishVersion">>,
<<"Judge", "AwaitApproval4">>,
<<"Judge", "Fail">>,
<<"PublishVersion", "Idle">>,
<<"PublishVersion", "AwaitApproval4">>,
<<"PublishVersion", "Fail">>,
\* Fase 6 · Regeneration
<<"Idle", "RequestChange">>,
<<"RequestChange", "Invalidate">>,
<<"Invalidate", "RegenerateAffected">>,
<<"RegenerateAffected", "Validate">>,
<<"Idle", "Branch">>
}
```

Los invariantes interrogan al grafo, no a una guarda escrita para que pasen

```

PublishVersion ==
  /\ /\ Mueve("PublishVersion", "Idle")
  /\ versiones' = Append(versiones, aprobados)
  /\ publicoSinValidar' =
    publicoSinValidar /\ (aprobados \ validados # {})
  /\ ...  \* rechazo: vuelve a AwaitApproval4 o va a Fail
NoPublishUnvalidated == publicoSinValidar = FALSE

```

```

Repair ==
  /\ Mueve("Repair", "Validate")
  /\ intentos < MaxIntentos
  /\ intentos' = intentos + 1
RetriesBounded == intentos <= MaxIntentos

```

```

EscribiendoPasadaInicial ==
  /\ ~regenerando
  /\ pc \in {"WriteChapter", "Validate", "Extract", "Repair"}
ResumeIsExactlyOnce ==
  EscribiendoPasadaInicial => aprobados = 1 .. (capitulo - 1)
CorpusSelladoNoSeToca ==
  sellado => pc \notin {"Research", "FillGap"}

```

NoPublishUnvalidated · la puerta G5

PublishVersion no lleva guarda de validación a propósito: con ella, el invariante comprobaría la guarda recién escrita. Sin ella, TLC busca cualquier camino (reintentos, un gate que rehace, una regeneración a medias) que publique un capítulo fuera de validados.

El testigo es una **variable de historia**, publicoSinValidar, que nunca debe encenderse.

RetriesBounded · dos entradas, un contador

Repair tiene dos aristas de entrada, desde Validate y desde Extract, que comparten intentos. Por eso Extract es una acción propia y no un detalle de Validate: plegarla dejaría sin comprobar justo esta interacción.

ResumeIsExactlyOnce y CorpusSelladoNoSeToca

En la pasada inicial, los aprobados son exactamente los capítulos anteriores al que se escribe: ni un hueco ni un repetido, también tras una Caída en cualquier nodo.

Tras el sello nadie vuelve a Research ni a FillGap, los dos únicos nodos que escriben en mundo_*. TypeOK completa los cinco.

Toda generación termina: el Autor puede rehacer cuanto quiera, pero no infinitas veces

```

/* Las aprobaciones, por separado, para
   * poder exigirles equidad fuerte.
   */
AutorAprueba ==
  \/\ Aprobar("AwaitApproval", "Research")
  \/\ Aprobar("AwaitApproval2", "Plan")
  \/\ Aprobar("AwaitApproval3", "SealCorpus")
  \/\ Aprobar("AwaitApproval4", "Judge")

Spec == Init /\ [][Next]_vars
          /\ WF_vars(Next)
          /\ SF_vars(AutorAprueba)

/* Publica, ramifica o para con error.
   */
Termina == <>(pc \in {"Idle", "Fail", "Branch"})

/* La secuencia de versiones es append-only.
   */
PreviousVersionPreserved ==
  [][ \A i \in 1 .. Len(versiones) :
      /\ i \in 1 .. Len(versiones')
      /\ versiones'[i] = versiones[i] ]_vars

```

Por qué equidad fuerte y no débil

WF solo obliga a una acción **continuamente** habilitada. Aprobar en el gate de Writing no lo está: se deshabilita mientras dura el bucle de escritura que abre RehacerWriting. Bajo WF, el Autor que rehace para siempre es una traza admisible y Termina cae.

SF_vars(AutorAprueba) dice: si la aprobación está habilitada infinitas veces, acaba ocurriendo. Los otros tres gates no lo necesitan, porque rehacer una fase sin capítulos vuelve a habilitar la aprobación enseguida.

La comprobación no es vacía. Con WF_vars(AutorAprueba) en una copia, TLC viola Termina con la traza conocida.

El entorno está acotado

MaxCaídas, MaxReintentos y MaxCambiosLector limitan caídas, reintentos manuales y peticiones del lector. Sin la cota del lector, versiones crece en cada regeneración y el espacio no se agota.

PreviousVersionPreserved es una **propiedad de acción**: compara cada estado con el siguiente, sin variable de historia.

TLC agota los dos modelos, 12.650 y 4.425 estados distintos, sin una sola violación

CONSTANTES DEL MODELO

Constante	harness.cfg	harness_batch.cfg
NCapitulos	5	5
MaxIntentos	2	2
GatesActivos	TRUE	FALSE
MaxRechazosJuez	2	2
MaxCaidas	1	1
MaxReintentos	1	1
MaxCambiosLector	1	1

El modelo arranca con huecos = 2 (Init); el sistema real topa en 5 por ejecución de Plotting. La cota no cambia lo que se verifica: que el ciclo Plan ⇔ FillGap descuenta siempre.

COMPROBADO EN LAS DOS CONFIGURACIONES

- ✓ TypeOK
- ✓ NoPublishUnvalidated
- ✓ ResumelsExactlyOnce
- ✓ RetriesBounded
- ✓ CorpusSelladoNoSeToca
- ✓ PreviousVersionPreserved
- ✓ Termina

RESULTADO · TLC 2026.09.23 SOBRE JRE 21

Configuración	Generados	Distintos	Prof.	Tiempo
harness.cfg interactivo, SF sobre el Autor	22.349	12.650	71	1 s
harness_batch.cfg batch, sin Autor	7.029	4.425	66	1 s

12.650
estados distintos,
interactivo

4.425
estados distintos, batch

0
violaciones

Las cifras de formal/tla/README.md coinciden con la última auditoría (docs/.audit-state.json, bloque 8, 2026-09-25). TLC no ejecuta el código: explora todos los estados alcanzables del modelo. Corre en desarrollo, no en cada novela.

Por qué ahora termina. Antes el lector podía pedir cambios sin límite y versiones crecía sin fin; MaxCambiosLector hace finito el espacio y deja comprobar la liveness en el modelo de cinco capítulos.

TLC encontró tres errores de diseño que ninguna lectura del documento había visto

Fecha	Propiedad violada	Traza (resumen)	Qué se cambió
2026-09-23	ResumeIsExactlyOnce	35 estados. Se aprueban los 5 capítulos, el Autor pide rehacer en AwaitApproval4 y se vuelve a WriteChapter con aprobados = {1..5}.	El modelo. Rehacer Writing se trataba como pasada inicial. Nace RehacerWriting, que entra en modo regeneración.
2026-09-23	Termina (1.ª)	Ciclo Judge → AwaitApproval4 → Judge: el Autor aprueba y el juez vuelve a rechazar, sin fin.	Un tope que faltaba. rechazosJuez con MaxRechazosJuez y la arista Judge → Fail.
2026-09-23	Termina (2.ª)	Ciclo AwaitApproval4 → RehacerWriting → WriteChapter → ... → AwaitApproval4, indefinido.	La equidad. WF no bastaba; Spec pasa a SF_vars(AutorAprueba).

ACCIÓN TLA+ → IMPLEMENTACIÓN DEL NODO (NODOS)

Configure	intake:configure	AwaitApproval...4	gates:await_approval	Research	investigation:research
VerifyCorpus	investigation:verify	Plan	plotting:plan	FillGap	plotting:fill_gap
SealCorpus	plotting:seal	WriteChapter	writing:write	Validate	writing:validate
Extract	writing:extract	Repair	writing:repair	ApproveChapter	writing:approve
Checkpoint	writing:checkpoint	Judge	publication:judge	PublishVersion	publication:publish
Idle	graph:idle	RequestChange	regeneration:request	Invalidate	regeneration:invalidate
RegenerateAffected	regeneration:regenerate	Branch	graph.branch:fork	Fail	graph:fail

Rutas sin el prefijo storymaker. ni el sufijo .nodos. Los cuatro AwaitApproval comparten gates.nodos:await_approval. Caída y ResumeFromCheckpoint son el proceso que muere y storymaker continuar; Reintentar es storymaker reintentar; IdleRequest, regenerar.